Relatório de Auditoria de Segurança - Trampo

Auditoria estática orientada a cinco categorias de risco

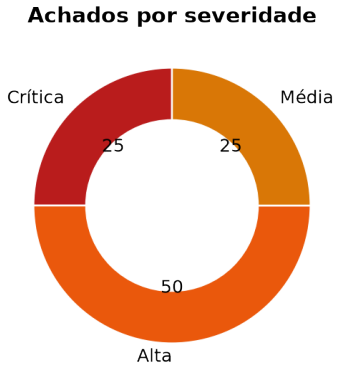
Data: 29/08/2026
Escopo: API Express, frontend React/Vite, app React Native, PostgreSQL, Docker Compose, Render, Vercel, scripts, documentação e histórico Git.

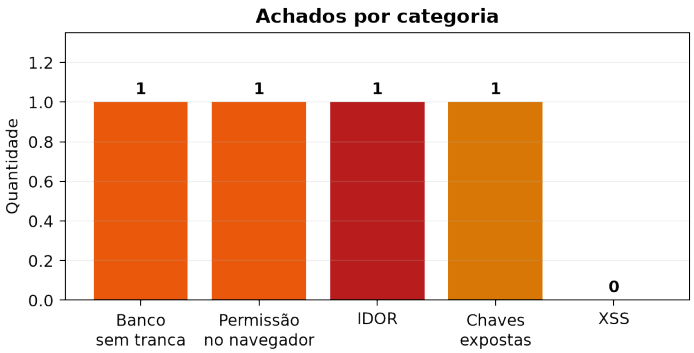
Metodologia: stack detectada pelo código real; isolamento mapeado para filtros manuais de ownership na API; autorização comparada entre gates React e rotas Express; handlers percorridos sistematicamente; segredos buscados no working tree, histórico e referências de bundle; sinks XSS buscados em web e mobile.

Resumo executivo

Foram verificados 4 achados acionáveis: 1 crítico, 2 altos e 1 médio. A categoria XSS foi auditada e não apresentou achado verificável.

Crítica	Alta	Média	Baixa	Pontos fortes
1	2	1	0	5





Pontos fortes e pontos fracos

Autenticação - JWT é verificado com jwt.verify; userFrom carrega o usuário por id e rejeita token inválido (server/index.js:99-108). Senhas de login são comparadas com bcrypt (server/index.js:143-150).

Fluxos críticos - accept-application, check-in, complete e chat-unlock verificam o usuário relacionado ao contrato/conversa dentro de transações (server/index.js:210-430).

Proteção contra UPDATE total - PATCH exige ao menos um filtro eq.* (server/index.js:496-498) e nomes de colunas são validados por IDENT (server/index.js:80, 466-468).

XSS - Nenhum innerHTML, dangerouslySetInnerHTML, v-html, eval ou new Function foi encontrado em src/mobile; renderização React usa escape padrão. Não há achado XSS verificável nesta revisão.

Histórico - Busca no histórico git por padrões de chaves e por trampo123 não encontrou um segredo adicional distinto dos defaults atuais; os defaults presentes no working tree continuam reportados acima.

Pontos fracos centrais - O CRUD genérico concentra leitura e escrita de tabelas sensíveis sem uma matriz completa de ownership. Isso torna a fronteira de segurança dependente de filtros opcionais enviados pelo cliente.

Achados detalhados

Linhas abaixo são referências exatas no working tree auditado; os trechos são resumidos no relatório para manter a tabela legível.

Severidade	Arquivo:linha	Descrição
● Alta	server/index.js:522-548	Leituras do CRUD genérico expõem dados de outras contas. A rota GET /api/data/:table aplica filtro de identidade apenas a contracts, proposals e escrow. users, payments, jobs, applications, conversations, messages, reviews e demais tabelas podem ser listadas por qualquer usuário autenticado, sem user_id, company_id ou vínculo de tenant.
● Alta	src/pages/DashboardAdmin.tsx:37-46; server/index.js:470-478	Dados administrativos são acessíveis sem papel admin. O frontend mostra o painel por meio de isAdmin e, no painel, solicita users, payments, jobs e escrow. Entretanto, a rota genérica só exige autenticação e bloqueio de conta; não há verificação admin para GET dessas tabelas.
● Crítica	server/index.js:481-488 e 491-519	CRUD genérico permite inserir e alterar objetos de terceiros. Para tabelas sem policy específica, POST aceita qualquer payload e PATCH aceita qualquer eq.<coluna>=valor. Não existe verificação de posse para jobs, applications, conversations, messages, payments, escrow e reviews; o corpo também pode escolher IDs e relações.
● Média	docker-compose.local.yml:9, 23-24; server/seed.js:22; README.md:35,57	Credenciais e senha padrão ficam hardcoded no repositório. O compose publica POSTGRES_PASSWORD=trampo_local_dev, DATABASE_URL com usuário/senha e JWT_SECRET=trampo_local_jwt_dev_change_me. O seed usa SEED_PASSWORD || trampo123 e a documentação divulga a senha das contas de demonstração.

Recomendações priorizadas

P1 - Eliminar ou fechar o CRUD genérico para escrita e criar políticas server-side de ownership por tabela; começar por payments, escrow, messages, conversations, jobs e applications.

P2 - Adicionar autorização admin no servidor para leituras administrativas e testes de acesso direto com usuários não-admin.

P3 - Remover defaults funcionais de compose/seed/docs, exigir segredo de seed e rotacionar credenciais compartilhadas.

P4 - Manter os controles verificados: JWT/bcrypt, transações nos fluxos críticos, allowlist de colunas e React sem sinks HTML perigosos; adicionar testes de regressão de isolamento.

ISSUES PARA O GITHUB

Issues completas, prontas para copiar e colar. Achados relacionados ao mesmo mecanismo foram mantidos em uma issue por risco.

```
--- ISSUE 1 ---
# [Segurança] Leituras do CRUD genérico expõem dados de outras contas
Labels sugeridas: `security`, `severity:alta`

## Descrição
A rota GET /api/data/:table aplica filtro de identidade apenas a contracts, proposals e escrow. users,
payments, jobs, applications, conversations, messages, reviews e demais tabelas podem ser listadas por
qualquer usuário autenticado, sem user_id, company_id ou vínculo de tenant.

## Evidência
`server/index.js:522-548`
Trecho: if (table === contracts/proposals/escrow) adiciona escopo; const where = clauses.length ? WHERE
... : ''; SELECT * FROM ${table} ${where}

## Impacto
Um usuário autenticado pode consultar PII, candidaturas, mensagens, pagamentos e dados operacionais de
terceiros. Explorável sempre que a API estiver acessível com uma sessão válida.

## Sugestão de correção
Definir uma política de leitura por tabela e montar o WHERE com a posse/participação do usuário no
servidor; negar tabelas sem regra explícita.

## Critérios de aceite
- [ ] Cada tabela tem uma regra de escopo server-side explícita
- [ ] Testes com dois usuários confirmam que listagem/busca não cruza contas
- [ ] Admin possui rota separada e auditada para dados globais

--- FIM ISSUE 1 ---
```

```
--- ISSUE 2 ---
# [Segurança] Dados administrativos são acessíveis sem papel admin
Labels sugeridas: `security`, `severity:alta`

## Descrição
O frontend mostra o painel por meio de isAdmin e, no painel, solicita users, payments, jobs e escrow.
Entretanto, a rota genérica só exige autenticação e bloqueio de conta; não há verificação admin para GET
dessas tabelas.

## Evidência
`src/pages/DashboardAdmin.tsx:37-46; server/index.js:470-478`
Trecho: DashboardAdmin: supabase.from(users/payments/jobs/escrow).select(*); API: if (!user) ... if
(user.blocked) ...; SELECT sem isAdmin

## Impacto
Qualquer usuário autenticado pode reproduzir as chamadas do painel e obter dados administrativos, ainda
que não veja a UI de admin. A ocultação do navegador não é uma barreira de autorização.

## Sugestão de correção
Exigir isAdmin(user) no servidor para endpoints administrativos e separar leituras operacionais de rotas
públicas/participativas.

## Critérios de aceite
- [ ] Usuário não-admin recebe 403 em cada leitura administrativa
- [ ] Testes chamam os endpoints diretamente sem depender da UI
- [ ] Admin continua obtendo somente os campos necessários

--- FIM ISSUE 2 ---
```

```
--- ISSUE 3 ---
# [Segurança] CRUD genérico permite inserir e alterar objetos de terceiros
Labels sugeridas: `security`, `severity:crítica`

## Descrição
Para tabelas sem policy específica, POST aceita qualquer payload e PATCH aceita qualquer
eq.<coluna>=valor. Não existe verificação de posse para jobs, applications, conversations, messages,
payments, escrow e reviews; o corpo também pode escolher IDs e relações.

## Evidência
`server/index.js:481-488 e 491-519`
```

```
Trecho: const policy = policies[table] || {}; POST: INSERT INTO ${table} (...); PATCH: UPDATE ${table}
SET ... WHERE ${where}; sem ownership para tabelas não listadas

## Impacto
Com um token válido, um atacante pode alterar status, vínculos, mensagens ou registros financeiros de
outro usuário, ou criar relações em nome de terceiros. O filtro obrigatório do PATCH evita UPDATE sem
WHERE, mas não verifica autorização do objeto.

## Sugestão de correção
Remover o CRUD genérico para dados sensíveis ou declarar allowlists de operação, campos e ownership por
tabela; aplicar a condição de posse na própria query e validar relações no servidor.

## Critérios de aceite
- [ ] PATCH/POST sem vínculo legítimo retorna 403
- [ ] Campos de ownership e saldo não são aceitos do cliente
- [ ] Testes cobrem cada tabela exposta e os casos path/query/body

--- FIM ISSUE 3 ---
```

```
--- ISSUE 4 ---
# [Segurança] Credenciais e senha padrão ficam hardcoded no repositório
Labels sugeridas: `security`, `severity:média`

## Descrição
O compose publica POSTGRES_PASSWORD=trampo_local_dev, DATABASE_URL com usuário/senha e
JWT_SECRET=trampo_local_jwt_dev_change_me. O seed usa SEED_PASSWORD || trampo123 e a documentação divulga
a senha das contas de demonstração.

## Evidência
`docker-compose.local.yml:9,23-24; server/seed.js:22; README.md:35,57`
Trecho: POSTGRES_PASSWORD: trampo_local_dev | JWT_SECRET: trampo_local_jwt_dev_change_me | const PASSWORD
= process.env.SEED_PASSWORD || trampo123

## Impacto
A condição de exploração é executar o compose/seed com esses defaults em ambiente acessível ou reutilizar
a senha em uma implantação. O servidor rejeita JWT_SECRET ausente apenas em produção, mas seed e
credenciais locais não têm validação equivalente.

## Sugestão de correção
Usar .env.example sem valores funcionais, exigir segredos fortes e SEED_PASSWORD em qualquer execução
não-local, rotacionar credenciais já compartilhadas e remover senhas reais da documentação.

## Critérios de aceite
- [ ] Nenhum valor funcional aparece em compose, scripts ou docs
- [ ] Startup/seed aborta diante de defaults ou produção sem segredo explícito
- [ ] Histórico e bundle são verificados após rotação

--- FIM ISSUE 4 ---
```